

When Simulated Annealing Selects a Fast Learner Instead of the Best Final Model

George D. Anderson II

Independent research

github.com/georgexyz · [BERT-BoSA](#)

September 19, 2026

Abstract

This project tests simulated annealing as a way to choose training hyperparameters for a BERT-style text classifier trained from scratch on AG_NEWS. The search changes learning rate, encoder dropout, and encoder depth. Each candidate trains for 300 batches and is ranked by validation cross-entropy loss. In the recorded run, the best candidate reduced this short-run loss from 1.4264 for the starting configuration to 0.9803. After three full training epochs, the selected configuration achieved 87.34% test accuracy, compared with 88.08% for the starting configuration in an earlier run made with a previous version of the search code. This result is consistent with a mismatch between the short search budget and the final training objective: a model that improves quickly may not finish strongest. Because the comparison uses single runs whose random-number state is not matched, it should be treated as a case study rather than a general conclusion about simulated annealing.

1. Research question

Can simulated annealing improve the final accuracy of a from-scratch BERT-style classifier by selecting its learning rate, dropout, and number of encoder layers? More specifically, does validation loss after a small, fixed training budget reliably identify the configuration that performs best after full training?

The distinction matters because the search evaluates candidates after only 300 training batches, while the chosen model is subsequently trained for three full epochs. Those two measurements reward related, but potentially different, properties.

2. System and data

The model is a transformer encoder with token, position, and segment embeddings. The segment embedding is retained for fidelity to BERT but carries no information here: classification inputs are single sentences, so every token receives segment 0. It uses the representation of the first (`<cls>`) token for four-class news topic classification. A classification head applies a fixed 0.1 dropout followed by a linear layer; unlike encoder dropout, this rate is not part of the search space. The model follows a BERT-style encoder architecture, but it has **no masked-language-model pretraining**; all weights are learned directly from the classification task. The implementation is in `models/bert.py` and `models/classification_head.py`.

The experiment uses AG_NEWS. The data pipeline reserves 5% of the training split for validation and builds a vocabulary only from the remaining training texts. Text is lowercased, split into word tokens, and truncated to fit a maximum sequence length of 256. The test split is used for final evaluation, not for selecting search candidates. See `utils/data_utils.py`.

The starting configuration has eight encoder layers, a learning rate of 0.0001, and encoder dropout of 0.2. Fixed settings include 512-dimensional embeddings, eight attention heads, a batch size of 32, and three final training epochs. These values are defined in `config.py`.

3. Search procedure

At each simulated annealing step, the algorithm perturbs one of three parameters: learning rate (multiplied by a factor in $[0.5, 2]$, clamped to $[1e-5, 1e-3]$), encoder dropout (shifted by up to plus or minus 0.1, clamped to $[0, 0.5]$), or encoder layer count (shifted by up to plus or minus 2 layers, clamped to $[1, 16]$). These bounds delimit what the search could have found. A fresh model is trained for 300 batches with the candidate configuration, then evaluated on up to 50 validation batches. The cost is mean validation cross-entropy loss. Candidate evaluations use a fixed random seed to reduce variation from initialization and training-batch order. The search evaluates the starting configuration plus 15 proposed candidates.

A lower-loss candidate is always accepted. A higher-loss candidate can also be accepted with probability

$$P(\text{accept}) = \exp\left(-\frac{L_{\text{candidate}} - L_{\text{current}}}{T}\right),$$

where T is the current temperature. Cooling is geometric, with the rate derived from the requested iteration count so that the temperature falls from 0.1 to 0.001 across the run; the final acceptance decision used $T = 0.00136$. This allows occasional moves to worse short-run configurations early in the search. The best loss seen, rather than merely the final accepted configuration, determines the configuration used for full training. The procedure is implemented in `optimization/simulated_annealing.py` and called from `train.py`.

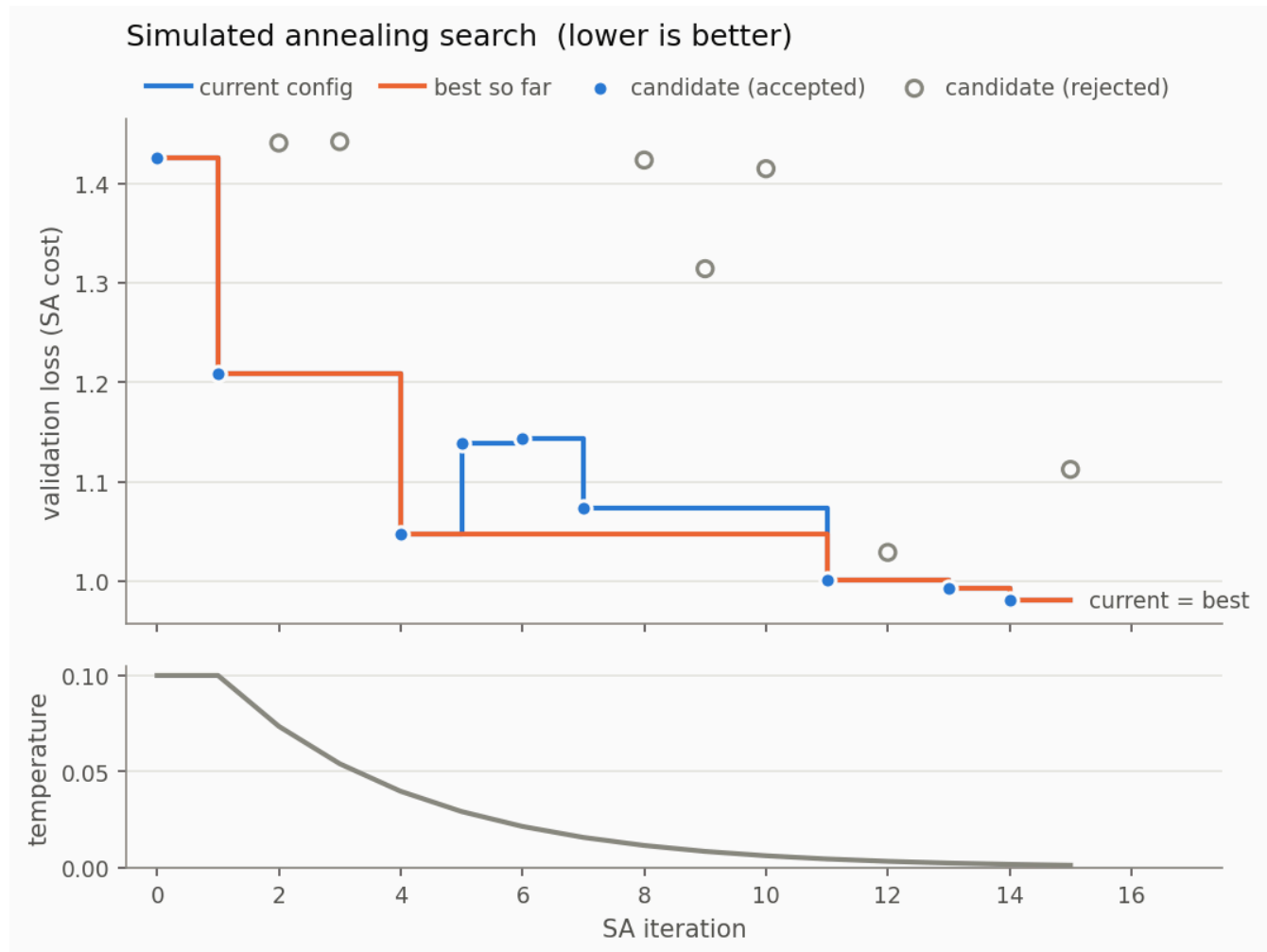
4. Results

The saved search trace records the following observations:

Configuration	Learning rate	Encoder dropout	Layers	Validation loss after 300 batches	Validation accuracy after 300 batches
Starting configuration	0.0001000	0.2000	8	1.4264	26.25%
Best by validation loss, iteration 14	0.0000737	0.1681	5	0.9803	60.44%

The best candidate's short-run validation loss is 0.4461 lower than the starting candidate's, a reduction of about 31.3%. Its validation accuracy is also higher by about 34.2 percentage points. These are **candidate evaluation** results, not final test results.

The search accepted worse-loss candidates at iterations 5 and 6, demonstrating the intended annealing behavior. Later, several candidates were rejected as the temperature fell. The complete observations are in [sa_log.csv](#), and the trajectory is shown below.



After the best configuration was selected, the project trained a fresh model for three epochs. That run reached **87.34% test accuracy**.

An earlier full run of the starting eight-layer configuration reached **88.08% test accuracy**, a difference of **0.74 percentage points**. That run was produced by a previous version of the search code, in which the cost was one minus validation accuracy rather than validation loss. Under that metric all 16 candidates scored identically (0.744375, chance accuracy for four balanced classes), so no candidate was ever preferred and the search returned its starting configuration unchanged. The eight-layer result is therefore a baseline for the untuned configuration, not the product of a successful search. Its log is preserved in [baseline_run.txt](#) and its degenerate trace in [baseline_accuracy_cost_sa_log.csv](#); that run reached

validation accuracies of 85.37%, 86.42%, and 87.73% across its three epochs. No checkpoint was retained for it.

5. Discussion

The search succeeded at its immediate task: it found a configuration with substantially lower validation loss after 300 training batches. That improvement did not translate into a higher reported final test accuracy. One plausible explanation is **short-horizon bias**. A five-layer encoder can potentially make faster early progress than an eight-layer encoder, while the deeper model may benefit more from longer training. The recorded results are consistent with this explanation, but they do not establish it on their own.

The experiment also shows why the choice of search metric matters, and the earlier run provides direct evidence. With cost defined as one minus validation accuracy, all 16 candidates scored exactly 0.744375: after a short budget each model still predicted a single class, so accuracy could not separate them. Every proposal was then accepted with probability $\exp(0) = 1$, and the search degenerated into a random walk that returned its starting point. Cross-entropy loss distinguishes the confidence of predictions, so it separates candidates at the same horizon. In the recorded run, loss differences are substantial even among candidates whose accuracies remain near chance.

The final comparison should be interpreted carefully, because the two runs differ by more than their configuration. Candidate evaluation reseeds the global random number generator before each training loop, and candidates consume differing amounts of randomness depending on their depth and dropout. The generator state entering final training therefore differs between the two runs, so weight initialization and batch order are not matched. Neither result includes variation across random seeds, and the two runs were produced by different versions of the search code. The 0.74-point gap describes these two runs; it is not an estimate of a reliable performance difference between search strategies.

6. Limitations and next experiment

The main limitation is the mismatch between the candidate budget and the final training budget. The search observes only 300 batches per candidate, while final performance is measured after three epochs. It also compares one recorded search run with one earlier starting-configuration result produced by different code and an unmatched random-number state. Different random seeds, initialization, and data order could change that small final accuracy gap.

A stronger follow-up experiment would use the same data split and several seeds to train both the starting and selected configurations for all three epochs. It would report mean test accuracy and variation across runs. To test the short-horizon explanation directly, the search could evaluate candidates at multiple budgets—for example, 300 batches, one epoch, and three epochs—and measure how well each budget's validation-loss ranking predicts final test accuracy. This would show whether a longer candidate evaluation improves selection enough to justify its extra compute cost.

7. Conclusion

This run demonstrates that simulated annealing can lower the short-run validation loss used to guide its search. It does not demonstrate an improvement in final classification accuracy. The most useful finding is the possible gap between **optimizing early training progress** and **selecting the best fully trained model**. Future runs with repeated seeds and longer candidate evaluations are needed to determine whether that gap explains the reported result.

Reproducibility notes

The saved example used 15 simulated annealing iterations, 300 training batches per candidate, and three epochs for the selected configuration. The default commands are:

```
python train.py --sa-iterations 15 --sa-train-batches 300 --epochs 3 --seed 42
python eval.py --checkpoint model.pt
python plot_sa.py --log sa_log.csv --output sa_trace.png
```

The training and evaluation entry points are `train.py` and `eval.py`. The recorded run used a Colab T4 with Python 3.13; the PyTorch version was not logged. The notebook that produced it is [BERT_BoSA_Colab.ipynb](#).

The baseline run in `baseline_run.txt` predates the current search code and is **not** reproducible with the commands above: it used the accuracy-based cost and a 100-batch candidate budget, and reproducing it requires commit `7be4018`.